Project Topic: Automated Bootstrapping Placement

> **How this fits the course project.** This is one of the pre-specified topics. The deliverables and deadlines are those of the Course Project page (proposal, milestone, final report, code repository). You are given the problem (Sections 1 to 3) and two blocks to solve (Section 4), each for every input level L_{init} from 0 to 14. *How* you find a placement is up to you; the techniques of this course all apply: state-space search over the edges with pruning (uninformed or A* with a bound on the remaining latency), a constraint-satisfaction formulation with the level constraints and forward checking, integer programming (covered in October), or local search over placements. The two blocks are provided as machine-readable graphs together with a baseline placement, plotting helpers and a reference scorer (Section 4.5).

1. Overview

In CKKS-style fully homomorphic encryption (FHE) secure inference, every multiplicative operation consumes *levels*. When the levels run out, a `Bootstrap` operation must refresh the ciphertext. Since bootstrapping is expensive, for a neural-network computation graph we want to decide **where to place the bootstrapping operations** so that the graph remains **level-correct** and the **estimated inference latency is minimised** under a **limited level budget**.

This project treats the network as a directed acyclic graph (DAG) and studies bootstrapping placement as an optimisation problem. The key difficulty is **skip connections**: real networks contain fan-out, fan-in, multi-input nodes and overlapping residual paths, and formulating them explicitly is a central challenge.

2. DAG structure

Let the graph be $G = (V, E)$. Each node v is an atomic network operator, including `Bootstrap`, with level cost d_v and latency cost $c_v(l)$, where l is its input level. Each edge e carries an encrypted tensor made of n_e ciphertexts (`#cipher`, or `#ct`). Let l_e be the level of edge e, and $l_{v,\text{in}}$, $l_{v,\text{out}}$ the input and output levels of node v.

Constraints.

1. The block input arrives at level L_{init}, an input condition of the problem (in Section 4, every value from 0 to 14). A `Bootstrap` restores $L_{\max}$ levels ($L_{\max} = 14$ in Section 4). There is no requirement on the level of the block's output edge.
2. For each non-`Bootstrap` node v: $l_{v,\text{out}} = l_{v,\text{in}} - d_v \geq 0$.
3. For each `Bootstrap` node v: $l_{v,\text{out}} = L_{\max}$.
4. For each edge $e = (u, v)$: $l_e = l_{u,\text{out}}$.
5. For a multi-input node such as `Add` with inputs i, j, all incoming edges must have compatible levels: $l_{\text{in}} = \min(l_{i,\text{in}}, l_{j,\text{in}})$ (the higher input is switched down for free).
6. A `Bootstrap` node may only be inserted where its input level is within the valid range given for it. All level variables are integers in $[0, L_{\max}]$. A `Bootstrap` or a `Modswitch` may also be placed on the block's input edge.

A solution is described by

- **depth**: $\text{depth} = \max_p \sum_{v \in p} d_v$, the level cost of the longest path p of the DAG (a `Bootstrap` has $d_v = 0$ and does not change it);
- **budget**: $\text{budget} = \#B \cdot L_{\max} + L_{\text{init}}$, where $\#B$ is the number of `Bootstrap` operations, counted per ciphertext;
- **utilisation**: $\text{utilisation} = \text{depth}/\text{budget}$.

Level costs of the basic FHE operations:

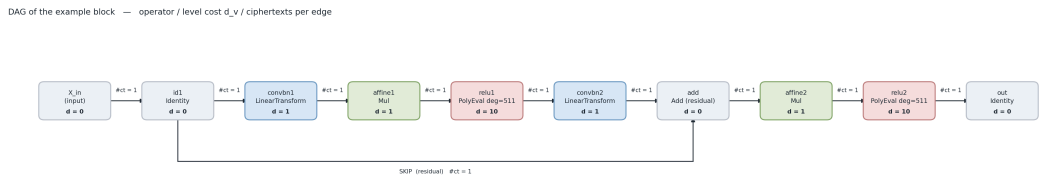
Operator	Level cost

`Mul`	1
`Add`	0
`Bootstrap`	given input level $l_{v,\text{in}}$, outputs the refreshed ciphertext at $l_{v,\text{out}} = L_{\max}$; $\#B =$ `#cipher` of the edge

3. Example: problem formulation for a CNN block

Section 2 is the general form of the problem. This section instantiates it on one small block, so that it is clear what you are given, what you decide and what you should report. How to find the placement is up to you.

3.1 The block



DAG of the example block: operator, level cost d_v, and ciphertexts per edge.

10 modules, 10 edges: `convbn1` and `convbn2` are `LinearTransform`; `affine1` and `affine2` are `Mul`; `relu1` and `relu2` are `PolyEval` with `deg=511`; `add` is the only `Add`, with two inputs, the trunk and the skip connection `id1 → add`; `X_in`, `id1` and `out` are `Identity`.

3.2 Level cost and estimated latency

Module	**Level cost d_v**	**Latency $c_v(l)$ at input level l**
`LinearTransform`	1	$0.001 \cdot 144 \cdot l$
`Mul`	1	0
`PolyEval`, `deg=511`	$\log_2(511+1)+1 = 10$	0
`Add`	0	0
`Identity`	0	0
`Bootstrap`	refresh to $l_{v,\text{out}} = L_{\max}$	$3.41\, e^{0.18 L_{\max}} + 4.81$, per ciphertext

A `Bootstrap` may be inserted on any edge: it spends no level and refreshes its output to $L_{\max}$.

3.3 What you decide, and what you optimise

$$\text{decide } x_e \in \{0,1\} \quad \text{for every edge } e \qquad (x_e = 0:\ \text{none},\ x_e = 1:\ \text{Bootstrap on})$$

$$\text{minimise } f(x) = \sum_{v \in V} c_v(l_{v,\text{in}}) + \sum_{e\,:\,x_e = 1} n_e \cdot t_{\text{boot}}$$

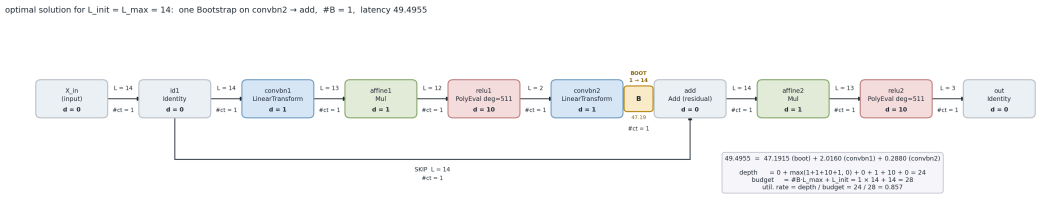
$$\text{subject to the level constraints of Section 2}$$

L_{init} and $L_{\max}$ are part of the input of the problem, $f(x)$ is the estimated latency of the block, and every placement that satisfies the constraints is a candidate solution.

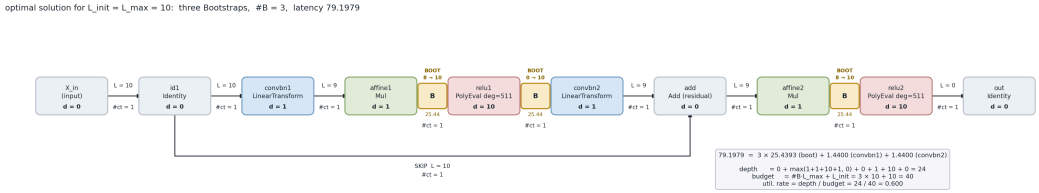
3.4 The optimal solutions

	$L_{\text{init}} = L_{\max} = 14$	$L_{\text{init}} = L_{\max} = 10$
estimated latency	**49.4955**	**79.1979**
$\#B$	1	3

bootstrapped edges	`convbn2 → add` (also optimal: `add → affine2`, `affine2 → relu2`)	`affine1 → relu1`, `relu1 → convbn2`, `affine2 → relu2`
depth / budget / utilisation	$24/28 = 0.857$	$24/40 = 0.600$



Optimal placement for $L_{\text{init}} = L_{\text{max}} = 14$: one `Bootstrap` on `convbn2 → add`, latency 49.4955.



Optimal placement for $L_{\text{init}} = L_{\text{max}} = 10$: three `Bootstrap`s, latency 79.1979.

3.5 What to report

- the placement: which edges carry a `Bootstrap` (hence $\#B$), and the level of every module;
- the estimated latency $f(x)$;
- **depth**, the level cost of the longest path of the DAG (a `Bootstrap` costs 0 level); here $1 + 1 + 10 + 1 + 1 + 10 = 24$;
- the budget $\#B \cdot L_{\text{max}} + L_{\text{init}}$ and the utilisation $\text{depth}/\text{budget}$, both defined in Section 2.

3.6 Reference implementation

`dag_solver_example.py` (with `dag_builder.py`, Section 4.5; both on Canvas under Files → Project → Automated Bootstrapping Placement) writes this example block down as a list of nodes and edges, scores a placement (`evaluate()`), enumerates all placements (`solve()`) and prints the table above. It is one way of writing the DAG and the latency table down, not a required structure; for the blocks of Section 4, enumeration is not an option.

4. Project: placement for Transformer blocks

You are given two components of an FHE Transformer block, an MLP block and an attention block, as the two figures below. You may hard-code each graph or write a builder that reads it from a configuration file.

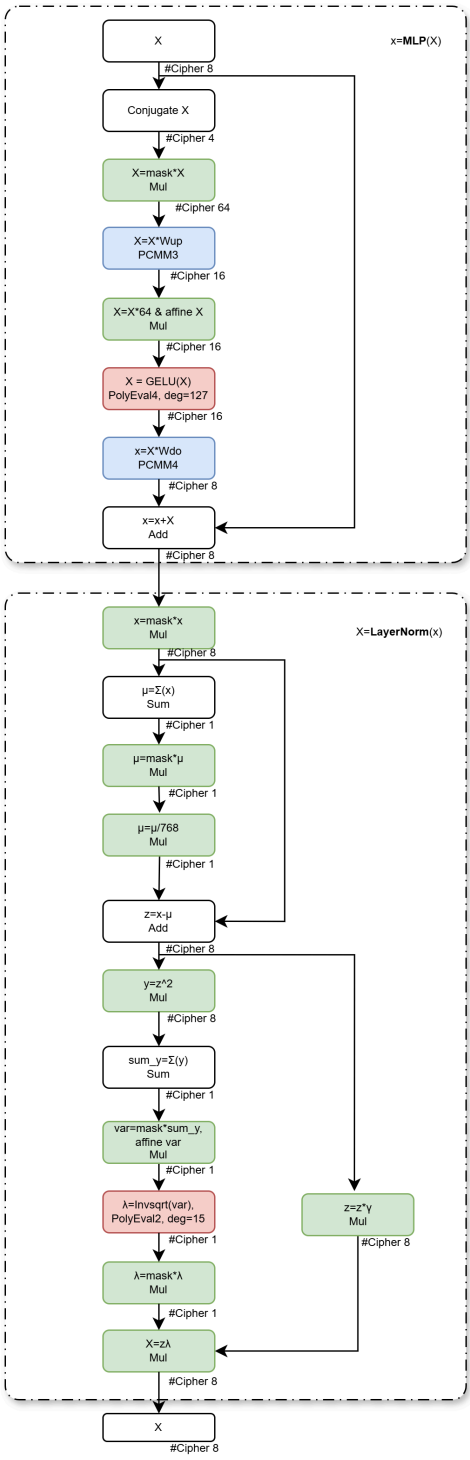
Level and latency costs for the operators of the Transformer are given as fitted functions of the input level, $c_{\text{op}}(l) = a_{\text{op}}\, l + b_{\text{op}}$, valid on the stated range of l:

Operator	Level cost	Latency $c_v(l)$ (ms)	Valid l	Description
`PCMM1`	2	$366.20\, l + 1797.71$	$2 \le l \le 14$	$V = X \cdot W_v$, $Q = X \cdot W_q$, $K = X \cdot W_k$
`PCMM2`	2	$104.64\, l + 2262.12$	$2 \le l \le 14$	$x = SV \cdot W_o$
`PCMM3`	2	$443.12\, l + 1581.48$	$2 \le l \le 14$	$X = X \cdot W_{up}$
`PCMM4`	2	$377.74\, l + 2095.11$	$2 \le l \le 14$	$X = X \cdot W_{down}$

`CCMM1`	4	$529.58\,l - 159.50$	$4 \le l \le 14$	QK = Q·K^{T}/√dim
`CCMM2`	2	$98.67\,l + 1566.84$	$2 \le l \le 14$	SV = S·V
`PolyEval1`	$\log_2(15+1)+1=5$	$142.64\,l - 455.72$	$5 \le l \le 14$	`Exp`, `deg=15`, in 8 ciphertexts
`PolyEval2`	$\log_2(15+1)+1=5$	$145.59\,l - 409.08$	$5 \le l \le 14$	`Invsqrt`, `deg=15`, in 1 ciphertext
`PolyEval3`	$\log_2(127+1)+1=8$	$562.27\,l + 819.37$	$8 \le l \le 14$	`Invsqrt`, `deg=127`, in 1 ciphertext
`PolyEval4`	$\log_2(127+1)+1=8$	$460.34\,l + 1311.36$	$8 \le l \le 14$	`GELU`, `deg=127`, in 16 ciphertexts
`Mul`	1	$0.0663\,l + 0.4192$	$1 \le l \le 14$	multiplication
`Add`	0	$0.0034\,l + 0.0379$	$0 \le l \le 14$	addition
`Bootstrap1`	$l \to L_{\max}$	1040.15 (constant)	$0 \le l \le 14$	refreshes the ciphertext to $l_{v,\mathrm{out}} = L_{\max}$; counted once per ciphertext (`#boot = #cipher`)
`Bootstrap2`	$l \to L_{\max} - 1$	1293.13 (constant)	$0 \le l \le 14$	refreshes to $l_{v,\mathrm{out}} = L_{\max} - 1$ and keeps the real part only, so it processes two ciphertexts at once (`#boot = #cipher/2`); for the given graphs it works only on edges with `#cipher` 8 or 16
`Modswitch`	$l - l'$	0	any	reduces the current level l to any lower level l', $0 \le l' < l$
`Conjugate`	0	$0.16\,l + 1.67$	$0 \le l \le 14$	halves the number of ciphertexts
`Conjugate back`	0	$0.60\,l + 2.02$	$0 \le l \le 14$	restores the number of ciphertexts
`Sum`	0	4.06 (constant)	$0 \le l \le 14$	collapses the ciphertexts into one
`Rotate`	0	0	any	rotates the slots of the ciphertexts (`Rotate SV` in the attention block); assumed free, like `Modswitch`

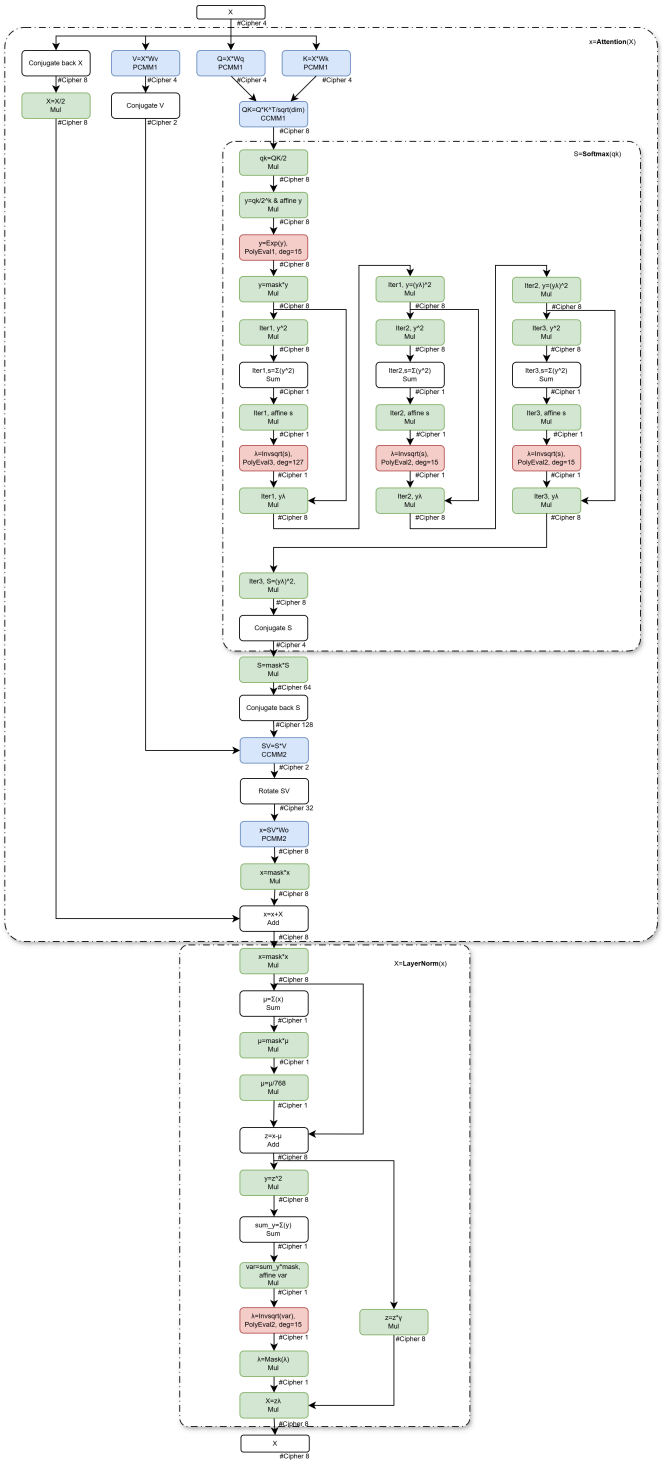
`Conjugate`, `Conjugate back`, `Sum` and `Rotate` are already placed in the graphs; you do not get to insert them. They cost no level, and their measured latency is a few milliseconds or zero, two to three orders of magnitude below a `PCMM` or a `Bootstrap`; they only change the number or the layout of the ciphertexts carried by an edge.

4.1 MLP block



The MLP block (with its LayerNorm). Node labels give the operation and operator type; edge labels give `#cipher`.

4.2 Attention block



The attention block (with its Softmax and LayerNorm). Node labels give the operation and operator type; edge labels give `#cipher`.

4.3 What you decide, and what you optimise

All outgoing edges of a node u carry the same tensor, with the same `#cipher` n_u: an operator inserted on the output of u is written **once**, can be read by any number of its successors, and is paid **once**. Section 3 decided per edge; this is the general form of that decision for graphs with fan-out. For every edge (u, v) you decide which inserted operators v reads after the output of u:

$$w_{u,v} \in \{\, \varnothing,\ B_1,\ B_2,\ M(\ell),\ B_1M(\ell),\ B_2M(\ell) \,\}, \qquad 0 \le \ell \le L_{\max}$$

B_1 / B_2 is a `Bootstrap1` / `Bootstrap2` (output level $L_{\max}$ / $L_{\max}-1$; the second only where `#cipher` is 8 or 16), $M(\ell)$ a `Modswitch` down to level ℓ. A second `Bootstrap`, a second `Modswitch`, or a `Modswitch` before a `Bootstrap` is never useful, so these six chains are all there is.

Two chains leaving the same node that begin with the same operators share them: they are **one** inserted node, written once and paid once. The cost therefore sums over the distinct inserted nodes, not over the edges:

$$\text{minimise } f = \sum_{v \in V} c_v(l_{v,\text{in}}) + \sum_{u \in V} \sum_{p \in P_u} \text{lat}(p)$$

$$\text{where lat}(p) = \begin{cases} n_u \cdot t_{\text{boot1}} & \text{if } p \text{ ends with } B_1 \\ \frac{n_u}{2} \cdot t_{\text{boot2}} & \text{if } p \text{ ends with } B_2 \\ c_{\text{mod}} & \text{if } p \text{ ends with } M(\ell) \end{cases}$$

$$\text{subject to the level constraints of Section 2}$$

where P_u is the set of distinct non-empty prefixes of the chains of the edges leaving u (each prefix is one inserted node); t_{boot1} is the cost of one `Bootstrap1`, which refreshes one ciphertext, and t_{boot2} the cost of one `Bootstrap2`, which refreshes a pair, so a tensor of n_u ciphertexts needs $n_u/2$ of them. A chain fixes the level of its edge: B_1 / B_2 / $M(\ell)$ gives $L_{\max}$ / $L_{\max}-1$ / ℓ, and an empty chain keeps $l_{u,\text{out}}$; $l_{v,\text{in}}$ then follows from the level constraints of Section 2. $L_{\max} = 14$ is fixed, and $c_{\text{mod}} = 0$ here: a `Modswitch` is a real operator with a cost, kept at 0 for this problem. L_{init} is an input condition: your method must produce a placement for every L_{init} from 0 to 14, and you report results over the whole range. f is the estimated latency of the block, and every placement that satisfies the constraints is a candidate solution.

4.4 What to hand in

- the problem you solved: the graph (modules, edges, `#cipher` on each edge) and the level cost and latency of every module;
- your problem formulation, your optimisation algorithm, and how you get a solution out of it;
- the best placement you found for each L_{init} from 0 to 14: a table or plot of the estimated latency against L_{init}, and for every value the numbers below:
  - the estimated latency $f(x)$;
  - the number of bootstraps $\#B = \#B_1 + \#B_2$, counted per ciphertext: a `Bootstrap1` node contributes the `#cipher` of the tensor it refreshes, a `Bootstrap2` node `#cipher/2`;
  - **depth**, the level-cost depth of the block;
  - **budget** $= \#B_1 \cdot L_{\max} + \#B_2 \cdot (L_{\max}-1) + L_{\text{init}}$ (a `Bootstrap1` gives back $L_{\max}$ levels, a `Bootstrap2` only $L_{\max}-1$);
  - **utilisation** $= \text{depth}/\text{budget}$;
- a figure of the placed block in the style of Section 3.4, for at least two values of L_{init}: the level of every edge, every operator you inserted, and the same numbers as above.

4.5 What is given: `dag_builder.py` and the baselines

`dag_builder.py` (on Canvas, Files → Project → Automated Bootstrapping Placement) is the handout. `get_mlp_empty()` and `get_attn_empty()` give the two blocks as *empty* DAGs: the modules with their operator type and level cost, and the `#cipher` of every edge. The file also holds the latency table of Section 4; `evaluate_dag()`, `depth_of()` and `budget_of()`, which score a

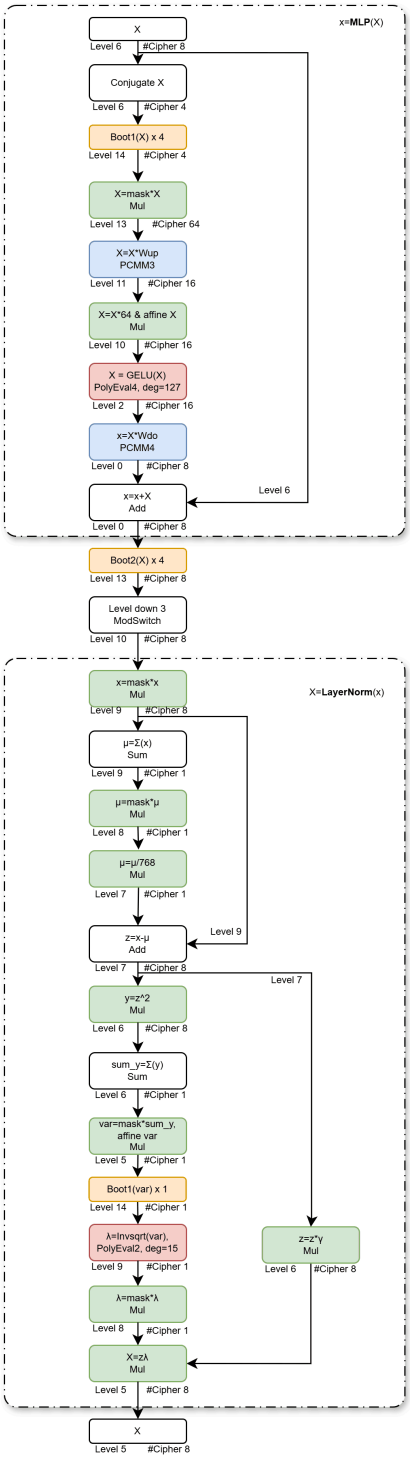
placement (the level of every node and edge, f, depth, budget, utilisation); and `visualize_empty_dag()`, which draws a DAG in the style of the figures above. `dag_solver_example.py` scores the CNN example of Section 3 through the same functions and is the template to copy.

You are expected to build your own model of the problem and your own optimiser. From the placement you get, build the *inserted* DAG (one real `Bootstrap1` / `Bootstrap2` / `Modswitch` node per inserted operator of Section 4.3, spliced in directly after the node whose output it reads; all three contribute 0 to the depth) and score it. The scoring functions of the handout are the reference implementation of the rules of this page: whatever scorer you use must reproduce the baseline numbers below before you optimise. If it does not, your reading of the rules differs from ours; if you think one of the functions is wrong, tell us, so that every solution is scored the same way.

The baselines. `get_mlp_baseline_inserted()` and `get_attn_baseline_inserted()` return each block with the `Bootstrap1` / `Bootstrap2` / `Modswitch` of the reference FHE implementation already in place, without any placement search, reading its L_{init}. The two figures below show them with the level of every edge ($L_{\text{init}} = 6$ for the MLP block, 9 for the attention block). In both blocks a `Bootstrap1` comes before the first level-consuming operator (right after the block input in the attention block, right after `Conjugate X` in the MLP block), so the same placement is valid for every L_{init} from 0 to 14 and its latency hardly depends on L_{init}. Its latency at each L_{init} is what you have to beat: feasible at the same $L_{\max}$ and cheaper, i.e. a smaller f. Do the MLP block (Section 4.1) first, then the attention block (Section 4.2).

MLP block: estimated latency 28,129.69 to 28,131.93 ms (28,130.65 ms at $L_{\text{init}} = 6$, the value of the figure); $\#B_1 = 5$, $\#B_2 = 4$; depth 26; budget $5 \cdot 14 + 4 \cdot 13 + L_{\text{init}} = 122 + L_{\text{init}}$.

Inserted operator	After node	Before node(s)	#cipher	Level in → out
`Bootstrap1`	`Conj_X_mlp`	`Mul_mask_mlp`	4	6 → 14
`Bootstrap2`	`Add_res_mlp`	`ModSwitch_down3_mlp`	8	0 → 13
`Modswitch`	`Boot2_X_mlp`	`x_mask2_mlp`	8	13 → 10
`Bootstrap1`	`mul_var_mlp`	`invsqrt_var_mlp`	1	5 → 14

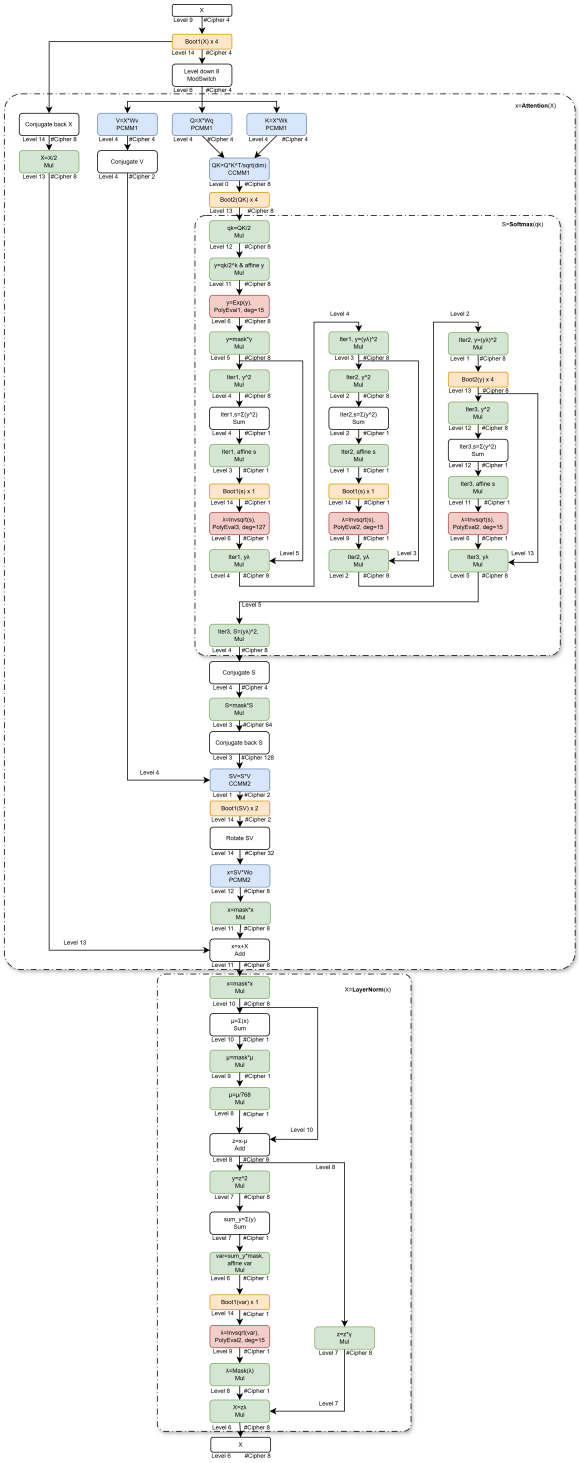


MLP block with the baseline placement, $L_{\text{init}} = 6$.

Attention block: estimated latency 53,558.17 ms for every L_{init}; $\#B_1 = 9$, $\#B_2 = 8$; depth 62; budget $9 \cdot 14 + 8 \cdot 13 + L_{\text{init}} = 230 + L_{\text{init}}$.

Inserted operator	After node	Before node(s)	#cipher	Level in → out
`Bootstrap1`	`X_in_attn`	`ModSwitch_down8_attn, Conj_back_X_attn`	4	9 → 14
`Modswitch`	`Boot1_X_attn`	`V_Wv_attn, Q_Wq_attn, K_Wk_attn`	4	14 → 6

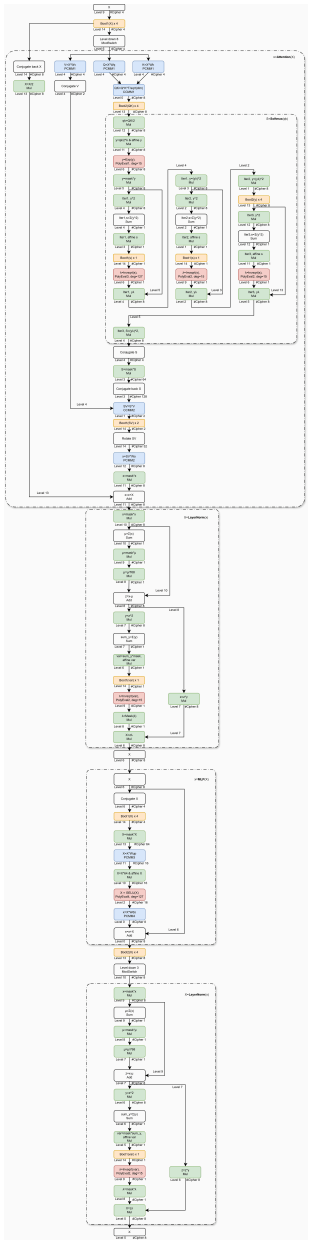
Bootstrap2	QK_mul_attn	qk_div2_attn	8	$0 \rightarrow 13$
Bootstrap1	iter1_affine_s_attn	iter1_invsqrt_attn	1	$3 \rightarrow 14$
Bootstrap1	iter2_affine_s_attn	iter2_invsqrt_attn	1	$1 \rightarrow 14$
Bootstrap2	iter2_y_sq2_attn	iter3_y_sq_attn, iter3_ylam_attn	8	$1 \rightarrow 13$
Bootstrap1	SV_mul_attn	rotate_SV_attn	2	$1 \rightarrow 14$
Bootstrap1	mul_var_attn	invsqrt_var_attn	1	$6 \rightarrow 14$



Attention block with the baseline placement, $L_{\text{init}} = 9$.

4.6 Optional: both blocks at once

Section 4.5 treats the two blocks on their own. To go further, solve them as one connected DAG: the same decisions as Section 4.3, but one placement now covers both blocks, and the MLP block reads the level the attention block hands over (the two joining edges are `X_out_attn → Conj_X_mlp` and `X_out_attn → Add_res_mlp`, `#cipher` 8 each), so an operator inserted on one side can pay off on the other. The placement to beat is then the whole attention + MLP block with the reference operators inserted, `get_baseline()`, shown below for $L_{\text{init}} = 9$: estimated latency 81,688.82 ms; $\#B_1 = 14$, $\#B_2 = 12$; depth 88; budget $14 \cdot 14 + 12 \cdot 13 + L_{\text{init}} = 352 + L_{\text{init}}$. This part is optional; it is not needed for the two blocks.



Baseline attention + MLP block, $L_{\text{init}} = 9$.